Prepared from the current local repository checkout at `/home/chris/cardvault`.

CardVault Decision Tables

This report is the compact version of the logic review.

Format used in each section:

- Formula
- If
- Then
- Result
- Notes / risk

---

1. Date and Deadline Math

1.1 Days remaining

Formula:

`days_remaining = target_date - today_in_calendar_days`

Decision table:

IF	THEN	RESULT
`dateString` is empty	return sentinel	`9999`
`dateString` is invalid	return sentinel	`9999`
`dateString` is valid	parse ISO date and compare to today at local midnight	integer day count

Interpretation:

IF `DAYS_REMAINING` IS	THEN	RESULT
`< 0`	treat as past due	deadline already passed
`= 0`	treat as current	due today
`> 0`	treat as future	due in X days
`= 9999`	treat as no usable deadline	ignored by many deadline-driven rules

1.2 Relative label formatting

Decision table:

IF	THEN	RESULT
`days >= 9999`	show no date	`None`
`days < 0`	show elapsed time	`X days ago`
`days = 0`	show same-day label	`Today`
`days = 1`	show next-day label	`Tomorrow`
`2 <= days <= 30`	show days	`in X days`
`31 <= days <= 60`	convert to weeks using `floor(days / 7)`	`in X weeks`
`days > 60`	convert to months using `floor(days / 30)`	`in X months`

---

2. Dashboard Totals

Source logic:

- cards are filtered first
- reward balances are summed directly from `rewardPrograms`

2.1 Active card counts

Formula:

- `active_cards = cards where status = active`
- `total_cards = count(active_cards)`
- `chris_cards = count(active_cards where cardholder = Chris)`
- `vanessa_cards = count(active_cards where cardholder = Vanessa)`

Decision table:

IF	THEN	RESULT
`card.status = active`	include in active-card pool	counted
`card.status != active`	exclude from active-card pool	not counted
active card belongs to Chris	add `1` to Chris count	person total increases
active card belongs to Vanessa	add `1` to Vanessa count	person total increases

2.2 Annual fee totals

Formula:

- `total_annual_fees = sum(active_card.annualFee)`
- `chris_fees = sum(active_card.annualFee where cardholder = Chris)`
- `vanessa_fees = sum(active_card.annualFee where cardholder = Vanessa)`

Decision table:

IF	THEN	RESULT
card is active	include `annualFee` in fee math	adds to total
card is closed	exclude	no contribution

2.3 Total points

Formula:

- `total_points = sum(reward_program.balance)`

Decision table:

IF	THEN	RESULT
reward program exists	read its `balance`	contributes to total
reward program balance is `0`	add zero	no effect

Risk:

- this does not always match the transfer engine’s loyalty-membership rules

2.4 Portfolio value

Formulas:

- `portfolio_value_min = sum(balance * conservative_cpp) / 100`
- `portfolio_value_max = sum(balance * optimistic_cpp) / 100`
- where:
- `conservative_cpp = conservativeCpp ?? cppRange[0]`
- `optimistic_cpp = optimisticCpp ?? cppRange[1]`

Decision table:

IF	THEN	RESULT
program has explicit `conservativeCpp`	use it	lower-bound valuation input
program lacks explicit `conservativeCpp`	use `cppRange[0]`	lower-bound fallback
program has explicit `optimisticCpp`	use it	upper-bound valuation input
program lacks explicit `optimisticCpp`	use `cppRange[1]`	upper-bound fallback

Result:

- value is represented in dollars after dividing cents-per-point by `100`

2.5 Cards needing action

Formula:

- count card if:
- `status != closed`
- and (`days_to_fee <= 60` or `keepCancelStatus = cancel`)

Decision table:

IF	THEN	RESULT
card is closed	exclude	no action count
card is active and fee date <= 60 days	include	action count +1
card is active and marked cancel	include	action count +1
both are true	still one card	count +1

2.6 Upcoming fees

Formula:

- include cards where:
- `annualFee > 0`
- `days_to_fee <= 120`
- sort ascending by `days_to_fee`

Decision table:

IF	THEN	RESULT
annual fee is `0`	exclude	not upcoming
annual fee > `0` and due within 120 days	include	appears in upcoming-fee list
annual fee > `0` and due after 120 days	exclude from this list	no row shown

2.7 Fees due in next 12 months

Formula:

- `upcoming_fees_12m = sum(annualFee for active cards where 0 <= days_to_fee <= 365)`

Decision table:

IF	THEN	RESULT
card is active and fee is due within 0 to 365 days	include fee amount	adds to 12-month forecast
fee date already passed	exclude	not counted
fee date beyond 365 days	exclude	not counted

2.8 Lifetime signup bonus value

Formula:

- `lifetime_signup_bonus_value = sum(enrollBonusValue ?? 0 for active cards)`

Decision table:

IF	THEN	RESULT
active card has `enrollBonusValue`	include value	total increases
active card has no `enrollBonusValue`	use `0`	no effect
card is closed	exclude	no contribution

3. Alert Engine

3.1 Severity

Formula:

- if `days < 60` -> red
- else if `days <= 120` -> amber
- else -> green

Decision table:

IF	THEN	RESULT
`days < 60`	mark urgent	`red`
`60 <= days <= 120`	mark medium urgency	`amber`
`days > 120`	mark low urgency	`green`

3.2 Annual fee alert

Decision table:

IF	THEN	RESULT
`annualFee > 0` and `nextFeeDate` exists	create alert	`annual_fee` alert
`annualFee = 0`	do not create alert	none
`nextFeeDate` missing	do not create alert	none

Formula shown in alert:

- description uses `formatCurrency(annualFee)`

3.3 Cancel deadline alert

Decision table:

IF	THEN	RESULT
card is active and `keepCancelStatus = cancel`	create cancel alert	`cancel_deadline`
card is closed	do not create cancel alert	none
card is active and not marked cancel	do not create cancel alert	none

Message branch:

IF	THEN	RESULT
`annualFee > 0`	use fee-sensitive wording	`Cancel before annual fee posts`
`annualFee = 0`	use generic wording	`Marked for cancellation`

3.4 Welcome bonus deadline alert

Decision table:

IF	THEN	RESULT
card is active, `subStatus` in active list, and `subDeadlineDate` exists	create alert	`bonus_deadline`
`subStatus` is terminal like `achieved` or `missed`	do not create alert	none
card is closed	do not create alert	none

Active SUB list:

- `targeting`
- `on_track`
- `not_sure`
- `worried`

Description formula:

IF	THEN	RESULT
`subSpendRequirement` exists	build spend/earn message	`Spend $X to earn Y points`
`subSpendRequirement` missing	use generic prompt	`Complete welcome bonus spend requirement`

3.5 First-year fee waiver expiry

Formula for first charge date:

- `first_charge_date = annualFeeFirstChargeDate`
- else `first_charge_date = dateOpened + 1 year`

Decision table:

IF	THEN	RESULT
`annualFeeWaivedYear1 = true` and `annualFee > 0`	evaluate fee-waiver rule	possible alert
explicit `annualFeeFirstChargeDate` exists	use explicit date	basis for countdown
explicit charge date missing but valid `dateOpened` exists	use `dateOpened + 1 year`	fallback countdown
`0 <= days_to_first_charge <= 45`	create alert	`fee_waiver_expiry`
outside that range	do not create alert	none

---

4. Recommendation Engine

4.1 High-fee keep warning

Decision table:

IF	THEN	RESULT
`annualFee > 400` and `keepCancelStatus = keep`	create review recommendation	`Consider canceling`
fee <= 400	no recommendation from this rule	none
marked cancel already	no recommendation from this rule	none

4.2 Cancel-action recommendation

Decision table:

IF	THEN	RESULT
`keepCancelStatus = cancel`	create cancel-action recommendation	action required
not marked cancel	none from this rule	none

Message branch:

IF	THEN	RESULT
`days_remaining >= 9999`	no reliable deadline	`No fee date set.`
`days_remaining > 0`	warn about future deadline	`Cancel within X days`
`days_remaining <= 0`	warn that fee may already have posted	past-due wording

4.3 Chase 5/24 status

Formula:

- `cutoff = today - 24 months`
- count cards where:
- `dateOpened` is valid
- `dateOpened > cutoff`
- `cardType != authorized_user`

Decision table:

IF	THEN	RESULT
card opened in last 24 months and not AU	count it	5/24 count +1
AU card	exclude	does not affect count
invalid or missing open date	exclude	does not affect count
count >= 4	create recommendation	Chase warning shown
count = 4	warning says approaching 5/24	caution
count >= 5	warning says at 5/24	wait on Chase

4.4 Companion Pass

Decision table:

IF	THEN	RESULT
recommendation engine runs	append generic companion-pass recommendation	always present

Risk:

- this is not conditional logic
- it behaves more like static guidance than computed advice

---

5. Chase Freedom Auto-Reclassification

Rule

Formula:

- `has_sapphire = any active Chase card where cardName contains "sapphire"`
- `target_program = has_sapphire ? chase_ur : cashback`

Decision table:

IF	THEN	RESULT
any active Chase Sapphire exists	Freedom cards become transferable points cards	`rewardsProgram = chase_ur`
no active Chase Sapphire exists	Freedom cards revert to cashback classification	`rewardsProgram = cashback`

A Freedom card is mutated only if:

- bank is Chase
- name contains `freedom`
- existing program is either `chase_ur` or `cashback`
- existing program differs from target

---

6. Application-Date and Eligibility Approximation

6.1 Application date resolution

Decision table:

IF	THEN	RESULT
confidence = `exact` and exact date exists	use exact date	precise date
confidence = `month` and month exists	use last day of that month	conservative date
confidence = `year` and year exists	use Dec 31 of that year	conservative date
confidence = `unknown` or data missing	return null	eligibility uncertain

6.2 Estimated SUB receipt date

Formula:

- `base_date = approvalDate ?? dateOpened`
- `estimated_sub_received = base_date + subDeadlineMonths + 30 days`

Decision table:

IF	THEN	RESULT
base date exists and `subDeadlineMonths` exists	compute estimate	approximate receipt date
base date missing	cannot compute	`null`
deadline months missing	cannot compute	`null`

6.3 Issuer reapplication rules

Hardcoded rule table:

ISSUER	RULE
Chase	48 months from SUB received
American Express	lifetime
Citi	24 months
Capital One	6 months
Bank of America	none documented
Barclays	24 months
Wells Fargo	none documented

Risk:

- this is static policy guidance, not live issuer terms

---

7. SUB Check-In Engine

7.1 Check-in eligibility

A card enters the due-checkin pool if all are true:

- `status = active`
- `subStatus` in:
- `targeting`
- `on_track`
- `not_sure`
- `worried`
- `subDeadlineDate` exists
- `days_remaining >= 0`
- `isCheckinDue = true`

Decision table:

IF	THEN	RESULT
card is closed	exclude	no check-in
terminal SUB state	exclude	no check-in
missing deadline	exclude	no check-in
overdue deadline	exclude	no check-in
active state and due	include	check-in prompt appears

7.2 Cadence logic

Decision table:

IF PREVIOUS STATE IS	THEN CADENCE IS	RESULT
no prior record	immediate	due now
`targeting`	30 days	next check-in after 30 days
`on_track`	`max(1, floor(days_to_deadline / 2))`	adaptive cadence
`not_sure`	7 days	weekly cadence
`worried`	3 days	high-frequency cadence
`achieved`, `achieved_early`, `missed`	stop	no more prompts

7.3 Dismiss and done actions

Decision table:

IF	THEN	RESULT
user dismisses with new state	write local record and update card `subStatus`	queue advances
user marks done	set local record to achieved, set `subReceived = true`, set receipt date	queue advances

8. Transfer and Reward Optimization

8.1 Balance source-of-truth rule

Decision table:

IF PROGRAM TYPE IS	THEN USE	RESULT
loyalty-membership program	`loyaltyMemberships[]`	derived balances
card-linked currency	`RewardProgram.balance`	household-level balance

Risk:

- some non-transfer screens do not fully follow this split

8.2 Best transfer recommendation

For each source currency with positive balance:

Formulas:

- `dest_cpp = conservative or optimistic destination CPP`
- `effective_cpp = dest_cpp * (toMultiplier / fromMultiplier)`
- `cash_cpp = source conservative/optimistic CPP`
- `transfer_value = round(balance * effective_cpp / 100)`
- `cash_value = round(balance * cash_cpp / 100)`

Decision table:

IF	THEN	RESULT
source program has `balance > 0` and transfer routes	evaluate routes	candidate recommendations
destination not found	skip route	no candidate
`effective_cpp <= cash_cpp`	reject route	not worth transferring
`effective_cpp > cash_cpp`	keep candidate	transfer beats cash-out
multiple valid routes exist	keep highest `transfer_cpp`	best route per source

Final result:

- one best route per source currency

- sorted by highest `transfer_value`

8.3 Household and person balances

Decision table:

IF FILTER IS	THEN	RESULT
`all`	sum all loyalty memberships by program	total loyalty balance
`chris`	sum Chris memberships only	Chris balance
`vanessa`	sum Vanessa memberships only	Vanessa balance
program is not a loyalty-membership program	use household `RewardProgram.balance`	unsplit currency balance

8.4 Monthly portfolio snapshot

Formulas:

- `loyalty_value = sum(loyalty_balance * cpp / 100)`
- `card_linked_value = sum(currency_balance * cpp / 100)`
- `total_value = round((loyalty_value + card_linked_value) * 100) / 100`

Decision table:

IF	THEN	RESULT
snapshot for current month already exists	return unchanged	idempotent
no current-month snapshot	compute value and insert	snapshot created
more than 36 snapshots exist after insert	keep newest 36	capped history

8.5 Month-over-month delta

Formula:

- `mom_delta = latest.totalValue - previous.totalValue`

Decision table:

IF	THEN	RESULT
fewer than 2 snapshots	cannot compare	`0`
at least 2 snapshots	subtract previous from current	delta value

8.6 Joint strategy rule

Decision table:

IF POOLING TYPE IS	THEN	RESULT
`native_pool`	recommend household pooling	proactive strategy
`au_workaround`	recommend AU-based combining	proactive strategy
`fee_transfer` and both people have memberships and combined >= 100,000	use one primary saver	`primary_saver`
`fee_transfer` and both people have memberships and combined < 100,000	save separately	`parallel_savers`
`fee_transfer` but one person missing	no strategy returned	none

Formula:

- `combined_balance = chris_balance + vanessa_balance`

---

9. Referral Optimizer

Rule

Decision table:

IF	THEN	RESULT
no referral and no public elevated offer	no comparison available	`no_data`
referral exists and elevated does not	recommend referral	`use_referral`
elevated exists and referral does not	recommend public offer	`use_elevated_public`
both exist and `referralPoints >= elevated.points`	choose referral	`use_referral`
both exist and `referralPoints < elevated.points`	choose elevated public offer	`use_elevated_public`

Comparison formula:

- compares raw points only

Risk:

- does not adjust for:
- spend requirement
- statement credits
- annual fee differences
- approval odds

---

10. Balance Gate

10.1 Initial gate

Decision table:

IF	THEN	RESULT
user already has cards on this device	auto-ack gate	intro gate suppressed
no cards and gate not acked	show gate	onboarding-style warning shown

10.2 Monthly check-in

Decision table:

IF	THEN	RESULT
gate acked and month not checked in	show monthly balance check	check-in modal visible
month already checked in	suppress monthly prompt	no modal

10.3 Profile sync rule

Decision table:

IF	THEN	RESULT
profile says `carries_balance`, device has not synced that flag, and local disabled flag missing	set disabled + gate acked	restrictive mode enabled
user later confirms paying in full	remove disabled flag	access restored
user says need help	set disabled flag	restrictive mode enabled

Risk:

- this is partly device-local behavior, not fully centralized

---

11. Cloud Sync Rules That Affect Calculations

11.1 Scope selection

Decision table:

IF	THEN	RESULT
user has active household	sync in household scope	shared data
user has no household	sync in individual scope	personal data

11.2 Initial sync

Decision table:

IF	THEN	RESULT
local `cloud-synced` flag is false	merge local and cloud	initial migration
local `cloud-synced` flag is true	fetch cloud as source of truth	returning-device flow

Risk:

- current flag is device-level, not user-level

11.3 Realtime suppression

Decision table:

IF	THEN	RESULT
app writes to cloud	suppress realtime for 2 seconds	avoids self-echo

11.4 Household migration

Decision table:

IF	THEN	RESULT
scope changes individual -> household	copy individual data to household and delete old individual rows	household becomes source
scope changes household -> individual	copy current local data to individual scope	personal scope restored

---

12. Priority Audit Checklist

If you are trying to validate a suspicious number, use this sequence:

IF THE NUMBER LOOKS WRONG	THEN INSPECT	RESULT
total points looks wrong	compare `RewardProgram.balance` vs `loyaltyMemberships`	identify source mismatch
valuation looks too high/low	inspect CPP assumptions	identify valuation input issue
fee countdown looks wrong	inspect `nextFeeDate` and `calculateDaysRemaining` behavior	identify date issue
recommendation looks too blunt	inspect hard thresholds	identify policy-style rule
all totals seem off after edits or device change	inspect cloud sync state	identify stale-data issue

13. Executive Summary

The logic is mostly:

- sums
- thresholds
- date windows
- static valuation assumptions
- heuristic approximations

The most likely causes of incorrect-feeling output are:

- stale synced data
- mixed balance sources
- static CPP assumptions
- inferred dates
- hardcoded thresholds being mistaken for precise optimization